Assignment 1

GPU Microarchitecture and Compute Capability

Name: Arnav Singhal

MIS: 612203012

Div: 1; Batch: B3

1 Introduction

This assignment focuses on identifying the microarchitecture and compute capability of the graphics card used for CUDA development. For this, a Google Colab notebook with a T4 GPU runtime was used in place of a local installation. The CUDA Toolkit and NVIDIA driver are pre-installed in this environment.

This report explains what compute capability and microarchitecture mean in the context of CUDA-capable devices, presents the output of `nvcc` and `nvidia-smi`, and discusses in detail the specific microarchitecture and compute capability of the device that was identified.

2 Compute Capability and Microarchitecture

2.1 Microarchitecture

A microarchitecture describes the specific internal hardware design of a processor family: the number and grouping of cores, the memory hierarchy, the native instruction set, and the resulting performance and power characteristics. NVIDIA releases a new GPU microarchitecture roughly every one to two years, each identified by a codename (for example Kepler, Maxwell, Pascal, Volta, Turing, Ampere, Ada Lovelace, and Hopper). Every physical GPU model belongs to exactly one microarchitecture generation, and this generation determines which hardware features are available to CUDA programs running on it.

2.2 Compute Capability

Compute capability is a version number, expressed as `major.minor` (for example `7.5`), that NVIDIA assigns to each microarchitecture generation to indicate which hardware features and CUDA instructions the device supports. The major version identifies the core architecture generation, while the minor version indicates incremental improvements within that generation. CUDA source code is compiled against a target compute capability using the `nvcc -arch` flag, ensuring that the generated machine code (SASS) matches the instruction set actually available on the device; compiling for a compute capability higher than the device supports results in a kernel launch failure at runtime. The compute capability of a device can be queried programmatically via `cudaGetDeviceProperties`, or, as done in this assignment, inferred from the GPU model reported by `nvidia-smi` and cross-referenced against NVIDIA's published compute capability tables.

3 Identifying the GPU: nvcc and nvidia-smi

3.1 nvcc –version

`nvcc` is the CUDA compiler driver. Running `nvcc -version` reports the installed CUDA Toolkit version, i.e. release `12.8`.

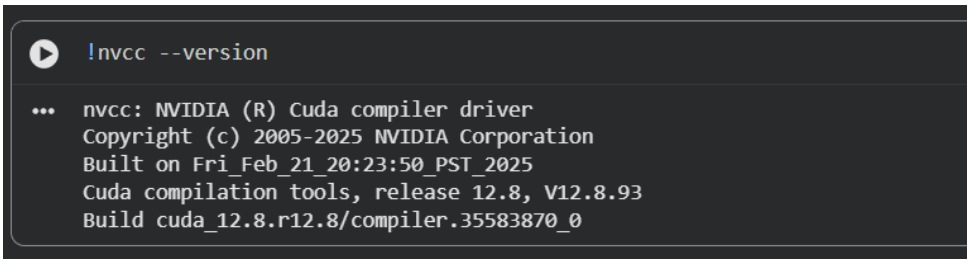

Figure 1: Output of nvcc –version confirming the installed CUDA Toolkit version.

3.2 nvidia-smi

`nvidia-smi` queries the NVIDIA driver directly and reports the GPU model, driver version, maximum supported CUDA version, and live memory/utilization status. Here, the device used is a `Tesla T4` with `15360 MiB` of memory, driver version `580.82.07`, and a maximum supported CUDA version of `13.0`. This is distinct from the toolkit version reported by `nvcc`: `nvidia-smi` reports the newest CUDA version the driver can support, while `nvcc` reports the toolkit version actually used for compilation.

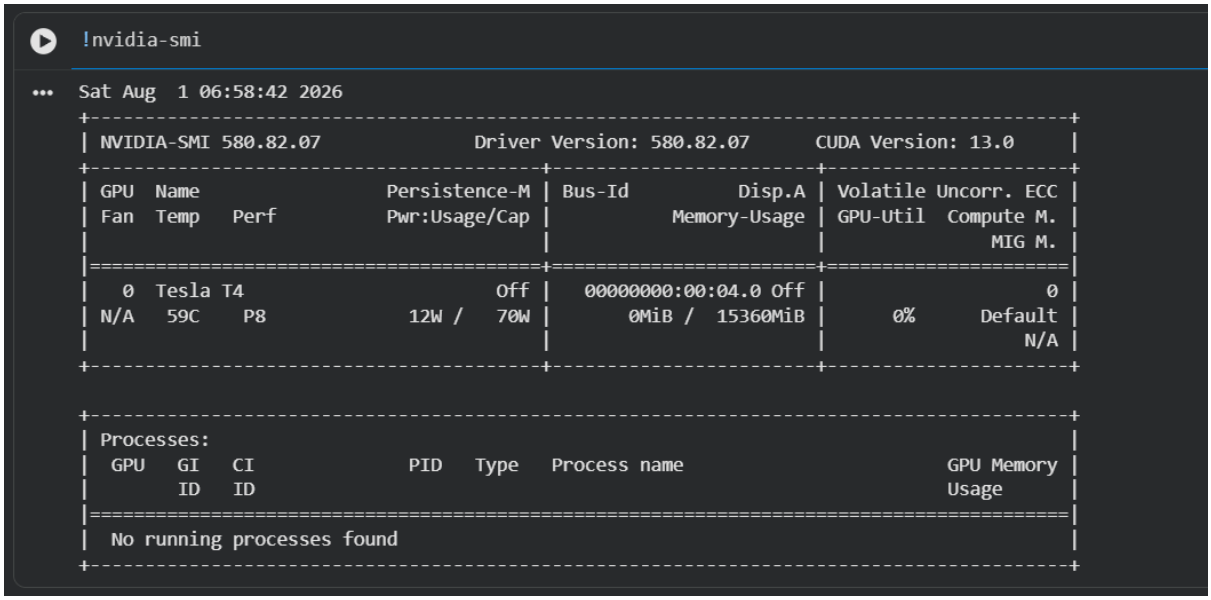

Figure 2: Output of `nvidia-smi` showing the Tesla T4 GPU, driver version, and memory status.

3.3 nvidia-smi -q

The `-q` (query) flag prints a detailed, verbose report from `nvidia-smi` instead of the default summary table, including per-GPU attributes such as product architecture, VBIOS version, PCIe generation/link width, ECC mode, clock speeds, power limits, and detailed memory usage. Running `nvidia-smi -q` on the allocated Tesla T4 confirmed the product architecture as `Turing` directly, corroborating the compute capability identified in Section 4.

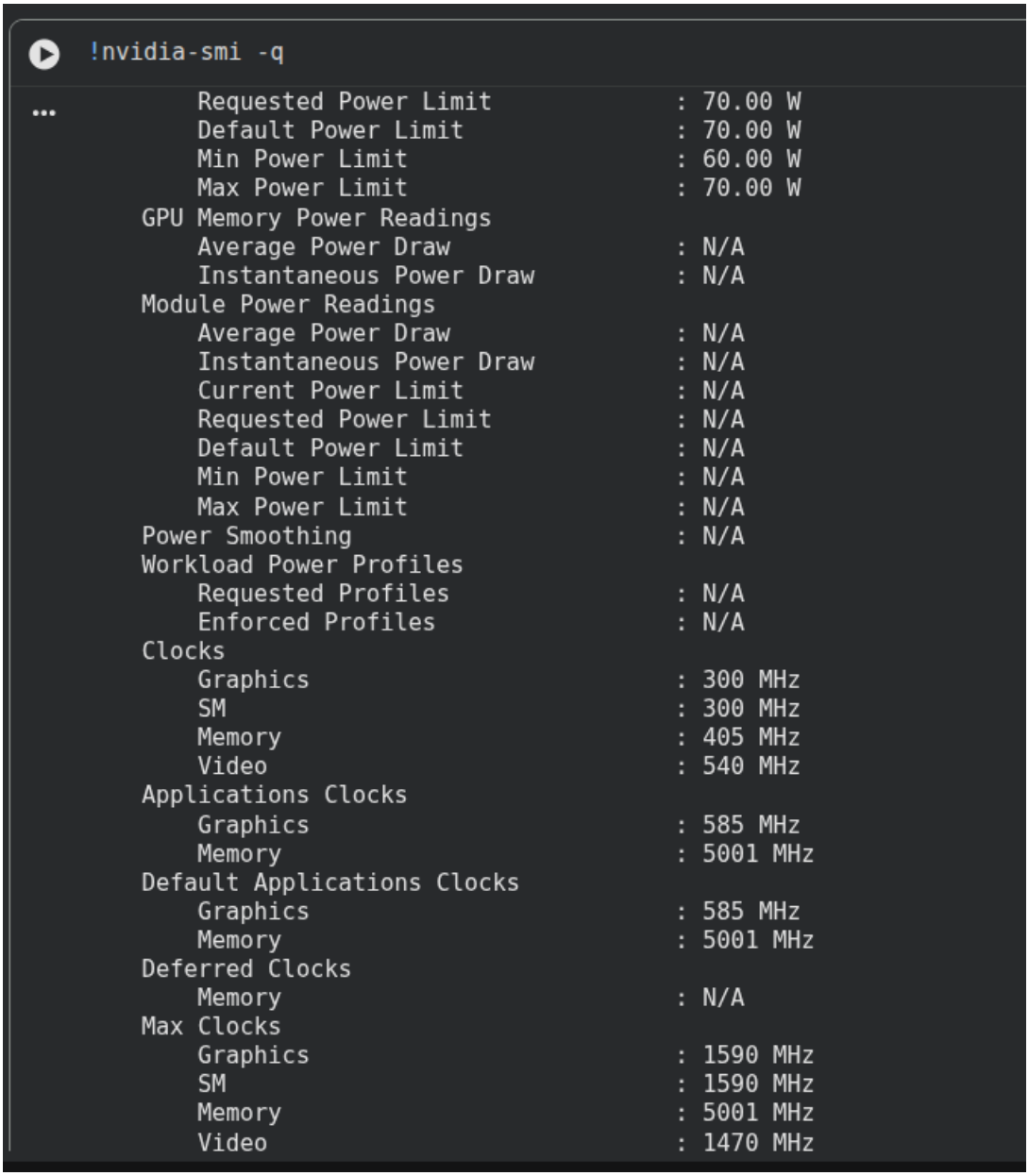

Figure 3: Output of `nvidia-smi -q` showing detailed attributes of the Tesla T4.

4 Microarchitecture and Compute Capability of the Device Used

The device used was an NVIDIA **Tesla T4**. Cross-referencing this model against NVIDIA's compute capability tables places it in the **Turing** microarchitecture generation with a **compute capability of 7.5**, which was subsequently used to target the compiler directly with the flag `-arch=sm_75` when compiling the sample kernel in Section 5.

4.1 Key Specifications of the Tesla T4 (Turing, sm_75)

Microarchitecture	Turing
Compute Capability	7.5 (`sm_75`)
Streaming Multiprocessors (SMs)	40
CUDA Cores per SM	64
CUDA Cores (Total)	2,560
Tensor Cores	320 (2nd generation)
RT Cores	40 (1st generation)
Memory	16 GB GDDR6
Memory Bus Width	256-bit
Form Factor	Single-slot, low-profile, low-power (70 W)
Typical Role	Cloud inference / general-purpose compute accelerator

- Turing was the first NVIDIA architecture to introduce dedicated **RT Cores** for real-time ray tracing, alongside a second generation of **Tensor Cores** for accelerated mixed-precision matrix operations used heavily in deep learning workloads.
- Compute capability 7.5 supports **independent integer and floating-point datapaths** within each Streaming Multiprocessor, allowing integer address and control-flow computation to proceed concurrently with floating-point math rather than contending for the same execution units.
- It supports newer CUDA features unavailable on older architectures, including improved warp-level primitives and mixed-precision Tensor Core operations, which determine what CUDA code and libraries can actually run on the device.
- The Tesla T4 variant of this architecture is specifically built as a low-power, single-slot inference and general-purpose accelerator. It offers a practical balance of compute capability, memory bandwidth, and power draw.

5 Verifying the Setup

To confirm that the identified compute capability was compatible with the toolkit and hardware, a minimal CUDA kernel was compiled directly against `sm_75` and executed. The kernel launches five GPU threads, each printing a message identifying its own thread index.

```
#include <stdio.h>

__global__ void helloFromGPU() {
    printf("Hello World from GPU thread %d!\n", threadIdx.x);
}

int main() {
    helloFromGPU<<<1, 5>>>();
    cudaDeviceSynchronize();
    return 0;
}
```

```
!nvcc -arch=sm_75 hello.cu -o hello
```

```
!./hello
```

Successful compilation and execution against `-arch=sm_75` confirms that the compute capability identified from `nvidia-smi` was correct and that the CUDA Toolkit reported by `nvcc` was able to target it without error.

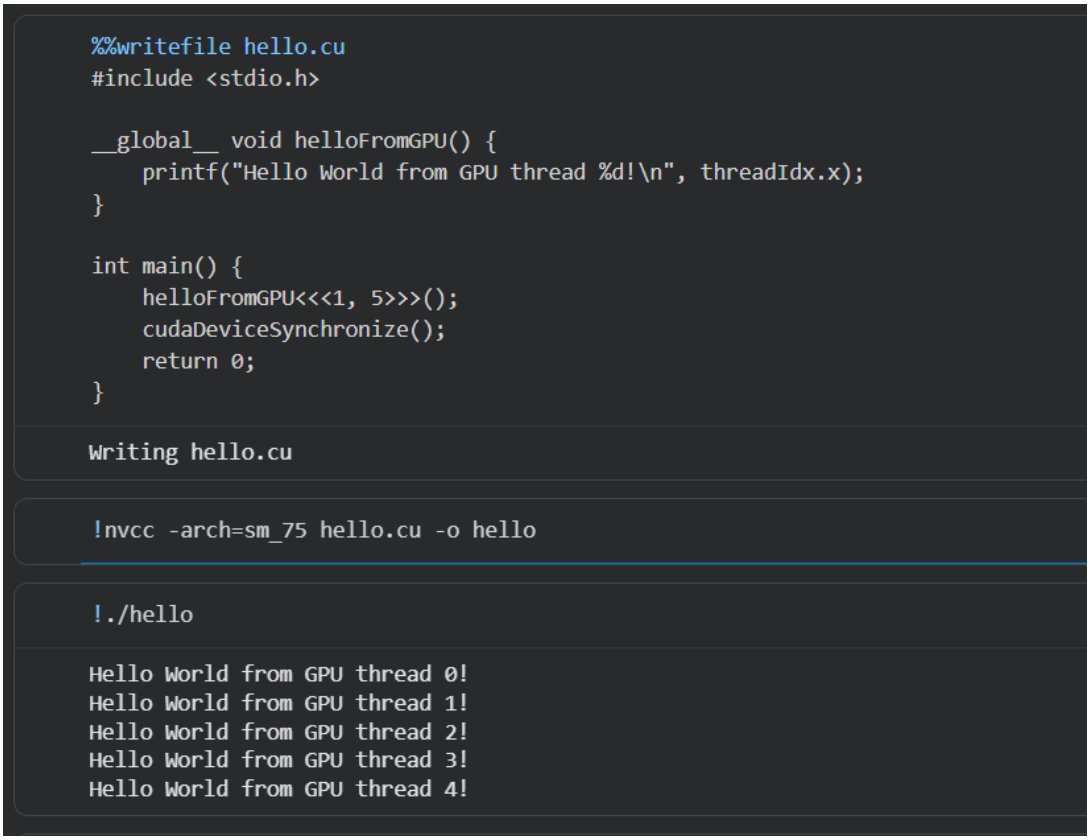

Figure 4: Output of the compiled `hello.cu` binary, showing all five GPU threads executing independently.

6 Conclusion

Using `nvcc -version` and `nvidia-smi` in a Google Colab runtime, the allocated GPU was identified as an NVIDIA Tesla T4 belonging to the Turing microarchitecture with compute capability 7.5. This report detailed the specifications of this microarchitecture and compute capability, and a CUDA kernel targeted at `sm_75` was successfully compiled and executed.